



Spring Boot Actuators

- Actuator brings production-ready features to application without implementing these features in the application.
- When application is in Production , you need to do Health Check , Auditing , Metrics gathering , Monitoring
- Actuator is mainly used to expose operational information about the application running in the production.
 - Health,
 - Metrics
 - Info
 - Env
 - Dump etc
- Actuator uses HTTP REST endpoints to enable us to interact with it.

hello-micro-service – Actuators Example

A)Develop the Hello MicroService

- 1) Copy demo2-rest-boot as **hello-micro-service**
- 2) **hello-micro-service** needs the following files

- a. MyBootApplication.java
- b. Hello.java
- c. HelloController.java
- d. application.properties
- e. pom.xml

- 3) Update pom.xml as follows.

```
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>hello-micro-service</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>hello-micro-service</name>
<description>Spring Boot Actuators</description>

<parent>
<groupId>org.springframework.boot</groupId>
```



```
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

4) Update application.properties

```
spring.application.name=hello-micro-service
server.port=54321

#Log Properties
logging.level.root=INFO
logging.level.com.jlcindia=DEBUG
logging.pattern.console=%-5level %logger{36} - %m %n
```

5) Write Hello.java

```
public class Hello {
    private int id;
    private String name;
    private String message;

    //Constrcutors
```



```
//Setters and Getters  
//toString()  
}
```

6) Write HelloController.java

```
@RestController  
public class HelloController {  
  
    @GetMapping(value="/myhello/{name}")  
    public Hello getMessage(@PathVariable String name) {  
        System.out.println("HelloController - getMessage()");  
  
        Hello hello =new Hello(101,name, "Hello Guys !!!" );  
        return hello;  
    }  
}
```

7) Write MyBootApplication.java

```
@SpringBootApplication  
public class MyBootApplication {  
  
    public static void main(String as[]) {  
        SpringApplication.run(MyBootApplication.class, as);  
    }  
  
}
```

B)Enabling Actuators in Project

8) Add the following dependency in pom.xml

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```

9) Start the Application and Open the URL

<http://localhost:12345/actuator>



10) By default, Exposes 1 endpoint called health.

Check the Endpoints here at <http://localhost:12345/actuator>

<http://localhost:12345/actuator/health>

11) Add the following property to enable required endpoints.

`management.endpoints.web.exposure.include=info,beans`

`management.endpoints.web.exposure.include=health,info,beans`

<http://localhost:12345/actuator/info>

<http://localhost:12345/actuator/health>

<http://localhost:12345/actuator/beans>

12) Add the following property to enable all endpoints.

`management.endpoints.web.exposure.include=*`

This enables all 13 endpoints

Check the Endpoints here at <http://localhost:12345/actuator>

<http://localhost:12345/actuator/info>

<http://localhost:12345/actuator/health>

<http://localhost:12345/actuator/beans>

<http://localhost:12345/actuator/env>

<http://localhost:12345/actuator/loggers>

<http://localhost:12345/actuator/metrics>

<http://localhost:12345/actuator/threaddump>

<http://localhost:12345/actuator/heapdump>

13) By default, all the endpoints are enabled except the shutdown endpoint.

You can enable or disable an actuator endpoint by setting the property as follows

Format:

`management.endpoint.<identifier>.enabled` to true or false

Here identifier is the endpoint.

Ex:

To Enable the shutdown endpoint, add the following to application.properties

management.endpoint.shutdown.enabled=true

To Disable the required endpoint, add the following to application.properties

management.endpoint.scheduledtasks.enabled=false

management.endpoint.caches.enabled=false

14) You can exclude the required endpoints from enabling them.

management.endpoints.web.exposure.exclude=conditions,configprops

15) /info endpoint

- By Default , No Information provided for the info endpoint.
- You need to configure required info in application.properties file

info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center

16) /loggers endpoint

- The loggers endpoint displays a list of all the configured loggers in your application with their corresponding log levels.

http://localhost:12345/actuator/loggers

- You can also view the details of an individual logger by passing the logger name in the URL like this

http://localhost:12345/actuator/loggers/{name}.

Ex:

http://localhost:12345/actuator/loggers/com.jlcindia
http://localhost:12345/actuator/loggers/com



17) /metrics endpoint

- The /metrics endpoint lists all the metrics that are available for you to track.

`http://localhost:12345/actuator/metrics`

- To get the details of an individual metric, you need to pass the metric name in the URL like this -

`http://localhost:12345/actuator/metrics/{MetricName}.`

Ex:

`http://localhost:12345/actuator/metrics/jvm.memory.max`

`http://localhost:12345/actuator/metrics/system.cpu.count`

18) /health endpoint

- The health endpoint checks the health of your application by combining several health indicators.
- Spring Boot Actuator comes with several predefined health indicators as follows
 - DataSourceHealthIndicator
 - DiskSpaceHealthIndicator
 - MongoHealthIndicator
 - RedisHealthIndicator
 - CassandraHealthIndicator etc.
- It uses these health indicators as part of the health check-up process.
- If your application uses Redis, the RedisHealthIndicator will be used as part of the health check-up.
- If your application uses MongoDB, the MongoHealthIndicator will be used as part of the health check-up.

Add the following in pom.xml for MongoDB

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

Add the following in application.properties for MongoDB

MongoDB Properties

`spring.data.mongodb.database=myjlcdb5`

`spring.data.mongodb.port=27017`

`spring.data.mongodb.host=localhost`

- If your application uses Any RDBMS, the DataSourceHealthIndicator will be used as part of the health check-up.

Add the following in pom.xml for MySQL

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>
```

Add the following in application.properties for MySQL

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

- You can Display detailed health information

```
management.endpoint.health.show-details = always
```

```
http://localhost:12345/actuator/health
http://localhost:12345/actuator/health/diskSpace
http://localhost:12345/actuator/health/db
http://localhost:12345/actuator/health/mongo
```

- You can develop Custom Health Indicators:

Steps to Write Custom Health Indicator

- 1) Write Health Indicator class by extending AbstractHealthIndicator
- 2) Override the doHealthCheck() method
public void doHealthCheck(Health.Builder builder) throws Exception
- 3) Write the required logic in doHealthCheck() method



@Component

```
public class MyCustomHealthIndicator extends AbstractHealthIndicator {
```

```
    @Override
```

```
    protected void doHealthCheck(Health.Builder builder) throws Exception {
```

```
        builder.up()
```

```
            .withDetail("app", "App is Running Successfully")
```

```
            .withDetail("error", "Nothing! I'm good.");
```

```
    }
```

```
}
```

- Now you can see Custom Health Indicators as follows

<http://localhost:12345/actuator/health/myCustom>

hello-micro-service –Boot Actuators Example

hello-micro-service: Files required

1. MyBootApplication.java	2. Hello.java
3. HelloController.java	4. MyCustomHealthIndicator.java
5. application.properties	6. pom.xml

1. MyBootApplication.java

```
package com.jlcindia.springboot;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
/*
```

```
 * @Author : Srinivas Dande
```

```
 * @company : Java Learning Center
```

```
 */
```

```
@SpringBootApplication
```

```
public class MyBootApplication {
```

```
    public static void main(String as[]) {
```

```
        SpringApplication.run(MyBootApplication.class, as);
```

```
    }
```

```
}
```



2. Hello.java

```
package com.jlcindia.springboot;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Hello {

    private int id;
    private String name;
    private String message;

    //Constrcutors
    //Setters and Getters
    //toString()
}
```

3. HelloController.java

```
package com.jlcindia.springboot;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@RestController
public class HelloController {

    @GetMapping(value="/myhello/{name}")
    public Hello getMessage(@PathVariable String name) {
        System.out.println("HelloController - getMessage()");

        Hello hello =new Hello(101,name, "Hello Guys !!!" );
        return hello;
    }

}
```



4. MyCustomHealthIndicator.java

```
package com.jlcindia.springboot;

import org.springframework.boot.actuate.health.AbstractHealthIndicator;
import org.springframework.boot.actuate.health.Health;
import org.springframework.stereotype.Component;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Component
public class MyCustomHealthIndicator extends AbstractHealthIndicator {

    @Override
    protected void doHealthCheck(Health.Builder builder) throws Exception {
        /*
        builder.up()
        .withDetail("app", "App is Running Successfully")
        .withDetail("error", "No Errors. App is doing good.");
        */
        builder.down()
        .withDetail("app", "App is Crashed")
        .withDetail("error", "Something Happened");
    }
}
```

5. application.properties

```
spring.application.name=hello-micro-service
server.port=12345

#Database Properties
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas

#JPA Properties
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# MongoDB Properties
spring.data.mongodb.database=myjlcdb5
spring.data.mongodb.port=27017
spring.data.mongodb.host=localhost
```



```
#Log Properties
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n

#management.endpoints.web.exposure.include=info,beans
#management.endpoints.web.exposure.include=health,info,beans

management.endpoints.web.exposure.include=*

management.endpoint.shutdown.enabled=true

management.endpoint.scheduledtasks.enabled=false
management.endpoint.caches.enabled=false

management.endpoints.web.exposure.exclude=conditions,configprops

management.endpoint.health.show-details = always

info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

6. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>hello-micro-service</artifactId>

<version>1.0</version>
<packaging>jar</packaging>

<name>hello-micro-service</name>
<description>Spring Boot Actuators</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>
```



```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Spring Boot Admin

- 1) Actuator is powerful and it is easy to use the endpoints from some other applications - you just make a simple REST call.
- 2) It is not so great when used by a human.
- 3) This is actually what Spring Boot Admin Does. It provides you UI layer on top of actuator endpoints with a few extra features on top.
- 4) Spring Boot Admin is not a core module provided by the Spring team, it was created by a company called Codecentric. Still, the code is publicly available on Github and it is free.
- 5) Spring Boot Admin comes in two parts –
 - ✓ Boot Admin Client
 - ✓ Boot Admin Server
- 6) The Server part contains the Admin user interface and runs independently from the monitored applications.
- 7) The Client part is in the monitored application, which registers with the Admin Server part.
- 8) If our application is down or not working properly, the monitoring server is still up and running. Now imagine you have multiple applications (such as Spring Boot microservices) and each of them can be running in multiple instances.
- 9) With traditional Actuator monitoring, this is hard as you need to access each of them separately and you need to keep track of how many instances and where are running.
- 10) With Spring Boot Admin, each instance of your monitored application (Client) registers with the Server after it starts. Then you have a single point (Admin Server), where you can check them all.



hello-micro-service

hello-micro-service: Files required

1. MyBootApplication.java	2. Hello.java
3. HelloController.java	4. application.properties
5. pom.xml	

1. MyBootApplication.java

```
package com.jlcindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
public class MyBootApplication {
    public static void main(String as[]) {
        SpringApplication.run(MyBootApplication.class, as);
    }
}
```

2. Hello.java

```
package com.jlcindia.springboot;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class Hello {

    private int id;
    private String name;
    private String message;

    //Constrcutors
    //Setters and Getters
    //toString()
}
```



3. HelloController.java

```
package com.jlcindia.springboot;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class HelloController {

    @GetMapping(value="/myhello/{name}")
    public Hello getMessage(@PathVariable String name) {
        System.out.println("HelloController - getMessage()");

        Hello hello =new Hello(101,name, "Hello Guys !!!" );
        return hello;
    }
}
```

5. application.properties

```
spring.application.name=hello-micro-service
server.port=5100

spring.boot.admin.client.url=http://localhost:5599

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n

management.endpoints.web.exposure.include=*

management.endpoint.scheduledtasks.enabled=false
management.endpoint.caches.enabled=false
management.endpoints.web.exposure.exclude=conditions,configprops

management.endpoint.health.show-details = always

info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```



6. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>hello-micro-service</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>hello-micro-service</name>
<description>This is my Hello MicroService</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.4.3</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>
</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



hai-micro-service

hai-micro-service: Files required

1. MyBootApplication.java	2. Hai.java
3. HaiController.java	4. application.properties
5. pom.xml	

1. MyBootApplication.java

```
package com.jlcindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
public class MyBootApplication {
    public static void main(String as[]) {
        SpringApplication.run(MyBootApplication.class, as);
    }
}
```

2. Hai.java

```
package com.jlcindia.springboot;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class Hai {
    private int id;
    private String name;
    private String message;

    //Constrcutors
    //Setters and Getters
    //toString()
}
```



3. HaiController.java

```
package com.jlcindia.springboot;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class HaiController {

    @GetMapping(value="/myhai/{name}")
    public Hai getMessage(@PathVariable String name) {
        System.out.println("HaiController - getMessage()");
        Hai hai =new Hai(901,name, "Hai Guys !!!" );
        return hai;
    }
}
```

5. application.properties

```
spring.application.name=hai-micro-service
server.port=9100

spring.boot.admin.client.url=http://localhost:5599

#Log Properties
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n

management.endpoints.web.exposure.include=*

management.endpoint.scheduledtasks.enabled=false
management.endpoint.caches.enabled=false
management.endpoints.web.exposure.exclude=conditions,configprops

management.endpoint.health.show-details = always

info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```



6. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>hai-micro-service</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>hai-micro-service</name>
<description>This is my Hai MicroService</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.4.3</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Setup Boot Admin Server

MyBootAdminServer : Working Steps:

- 1) Copy **hello-micro-service** as **MyBootAdminServer**.
- 2) Remove the following 3 classes from **com.jlcindia.springboot**
 - Hello.java
 - HelloController.java
 - MyCustomHealthIndicator.java
- 3) Re-name the Package to **com.jlcindia.mybootadmin**
- 4) Re-name the Boot Main class to **MyBootAdminServer**
- 5) Update **pom.xml** as follows
- 6) Update **application.properties**
- 7) Mark the BookMain class with **@EnableAdminServer** annotation.

BootAdminServer: Files required

1. MyBootApplication.java	2. application.properties
3. pom.xml	

1. MyBootApplication.java

```
package com.jlcindia.mybootadmin;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import de.codecentric.boot.admin.server.config.EnableAdminServer;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableAdminServer
public class MyBootAdminServer {

    public static void main(String as[]) {
        SpringApplication.run(MyBootAdminServer.class, as);
    }
}
```

2. application.properties

```
spring.application.name=MyBootAdminServer
server.port=5599
```



3. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>

<artifactId>MyBootAdminServer</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>MyBootAdminServer</name>
<description>This is MyBootAdminServer</description>

<properties>
<spring-boot-admin.version>2.4.3</spring-boot-admin.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.4</version>
</parent>

<dependencies>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-server</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-dependencies</artifactId>
<version>${spring-boot-admin.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
```



```
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Setup Boot Admin Client :

- 1) Choose Your Application.(**hello-micro-service** and **hai-micro-service**)
- 2) Add the following to pom.xml of **hello-micro-service** and **hai-micro-service**

```
<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.4.3</version>
</dependency>
```

- 3) Add the following to application.properties of **hello-micro-service** and **hai-micro-service**

```
spring.boot.admin.client.url=http://localhost:5599
```

Perform the Following Steps

- 1) Start **BootAdmin Server**
- 2) Open <http://localhost:5599>
- 3) Check Boot Admin web Console.
- 4) Start **hello-micro-service**
- 5) Check Boot Admin web Console.
- 6) Start **hai-micro-service**
- 7) Check Boot Admin web Console.



APPLICATIONS

2

INSTANCES

2

STATUS

all up



UP

✓ hai-micro-service
1m <http://SrinivasDande:9100/>

✓ hello-micro-service
1m <http://SrinivasDande:5100/>

8) Stop **hello-micro-service**

9) Configure 3 Instances for **hello-micro-service**

10) Start Instance 1 of **hello-micro-service** with port number - 5100

11) Start Instance 2 of **hello-micro-service** with port number - 5200

12) Start Instance 3 of **hello-micro-service** with port number - 5300

13) Stop **hai-micro-service**

14) Configure 3 Instances for **hai-micro-service**

15) Start Instance 1 of **hai-micro-service** with port number - 9100

16) Start Instance 2 of **hai-micro-service** with port number - 9200

17) Check Boot Admin web Console



APPLICATIONS

2

INSTANCES

5

STATUS

all up



UP

✓ hai-micro-service
1m 2 instances

✓ hello-micro-service
1m 3 instances